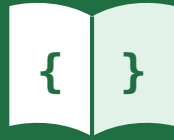

Modül 20 — Docker ve Deployment

Ders Notları

Spring Boot Edu



2026-09-25

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Bir Image Katmanlardan Oluşan Bir Yığındır	2
2.2 Liveness ve Readiness	2
2.3 Twelve-Factor Uygulama	3
3. Adım Adım Örnekler	3
3.1 Uygulama	3
3.2 Çok Aşamalı Bir Dockerfile	4
3.3 Buildpacks: Dockerfile Olmadan Bir Image	5
3.4 Health Probe'ları ve Zarif Kapanış	6
3.5 Ortamdan Yapılandırma	7
3.6 Docker Compose: Uygulama ve Veritabanı	8
4. Sık Yapılan Hatalar ve En İyi Pratikler	9
5. Özet	9
6. İleri Okuma	10

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Katmanlı jar, jlink ile yapılmış küçük bir JRE ve root olmayan bir kullanıcıyla çok aşamalı bir Dockerfile yazmak
- Dockerfile olmadan, Cloud Native Buildpacks ile (spring-boot:build-image) bir image build etmek
- Uygulamayı ve veritabanını Docker Compose ile birlikte çalıştırmak
- Liveness ve readiness probe'larını yapılandırmak ve farkı açıklamak
- docker stop'un çalışan istekleri bozmaması için zarıfçe (graceful) kapanmak
- Tek bir image'ı ortam değişkenleriyle her ortam için yapılandırmak (12-factor)

Ön koşullar: Modül 15 (Actuator, health) · **Tahmini süre:** 4 saat · **Docker gerekir**

2. Kavramlar

2.1 Bir Image Katmanlardan Oluşan Bir Yığındır

Bir Dockerfile'daki her komut (COPY, RUN) bir katman oluşturur. Docker katmanları önbellege alır ve altlarında hiçbir şey değişmediğinde onları yeniden kullanır. Bir Spring Boot fat jar tek bir dosyadır: Kodun bir satırını değiştirin, jar'ın tamamı (60 MB bağımlılık dahil) yeni bir katman olur. **Katmanlı bir jar** ile bağımlılıklar, Boot loader ve uygulama ayrı katmanlardır. Bir kod değişikliğinden sonra yalnızca küçük uygulama katmanı yenidir.

Katman	Değiştigi zaman	Tipik boyut
dependencies	bir kütüphane sürümü değiştiğinde	onlarca MB
spring-boot-loader	Boot yükseltildiğinde	< 1 MB
snapshot-dependencies	bir SNAPSHOT kütüphanenin her build'inde	küçük
application	her kod değişikliğinde	KB'lar ile birkaç MB

2.2 Liveness ve Readiness

Probe	Soru	Başarısız olduğunda platform ...	İçermesi gereken
Liveness	Süreç onarılamayacak şekilde bozuk mu?	container'ı yeniden başlatır	yalnızca uygulamanın kendi durumu
Readiness	Şu anda trafik alabilir mi?	istek göndermeyi durdurur (yeniden başlatma yok)	istekleri karşılamak için ihtiyaç duyduğu bağımlılıklar

Klasik hata, liveness'ta bir veritabanı kontrolüdür. Veritabanı çöktüğünde her örnek yeniden başlatılır. Bu veritabanına yardım etmez ve kesintiyi kötüleştirir.

2.3 Twelve-Factor Uygulama

"Twelve-Factor App", platformlarda iyi çalışan uygulamaların nasıl yazılacağını tarif eder. Bu modülde faktörlerden üçü en önemlidir:

- **III. Config:** Ortamlar arasında değişen yapılandırma image'dan değil ortamdan gelir.
- **IX. Disposability:** Hızlı başlangıç ve zarif kapanış.
- **XI. Logs:** Loglar stdout'a gider ve platform onları toplar (Boot bunu varsayılan olarak yapar).

3. Adım Adım Örnekler

Önce uygulamayı her zamanki gibi çalıştırın:

```
./mvnw -pl modules/20-docker-deployment/lesson spring-boot:run
```

```
=== readiness: ACCEPTING_TRAFFIC (shop "Bookstore") ===
probes: http://localhost:8080/actuator/health/liveness and /readiness
```

3.1 Uygulama

PostgreSQL üzerinde küçük bir API (/api/books), yapılandırma ve host adı (/api/shop) ve kapanış örneği için yavaş bir endpoint (/api/slow?millis=...). Bkz. [requests.http](#).

i Not

SPEC karar 12: Paketo buildpacks'te henüz Java 27 çalışma ortamı yok. Bu yüzden bu modül `maven.compiler.release=25` ile derlenir. Aynı jar Dockerfile'ımızın JRE 27'sinde ve Buildpacks image'ının JRE 25'inde çalışır.

3.2 Çok Aşamalı Bir Dockerfile

Önce jar'ı, sonra image'ı build edin:

```
./mvnw -pl modules/20-docker-deployment/lesson package
docker build -t springbootedu/docker-deployment:jlink
↪ modules/20-docker-deployment/lesson
```

İlk aşamada tam bir JDK vardır. Jar'ı katmanlara ayırır ve yalnızca uygulamanın ihtiyaç duyduğu JDK modülleriyle bir çalışma ortamı build eder:

```
# --- Stage 1: a full JDK 27 to take the jar apart and to build a runtime
FROM amazoncorretto:27-alpine AS builder
# jlink --strip-debug calls objcopy, which is part of binutils
RUN apk add --no-cache binutils
WORKDIR /builder
COPY target/*-SNAPSHOT.jar application.jar
# one folder per layer: dependencies change rarely, the application often
RUN java -Djarmode=tools -jar application.jar extract --layers --destination extracted
# which JDK modules does the application really use?
RUN jdeps --ignore-missing-deps --print-module-deps --recursive --multi-release 27 \
    --class-path 'extracted/dependencies/lib/*' extracted/application/application.jar
    ↪ > modules.txt
# a JRE with only these modules
RUN jlink --add-modules "${cat modules.txt}" \
    --strip-debug --no-man-pages --no-header-files --compress=zip-6 --output /jre
```

jdeps jar'ları analiz eder ve modülleri yazdırır (burada `java.base`, `java.sql`, `java.management`, ... — JDK'nın 70 modülünden yaklaşık 12'si). Ardından jlink tam olarak bu modüllerle bir JRE build eder.

İkinci aşama yalnızca bu JRE'yi ve katmanları içerir:

```
# --- Stage 2: only the JRE and the application, on plain Alpine
FROM alpine:3.22
RUN addgroup -S spring && adduser -S spring -G spring
ENV JAVA_HOME=/opt/jre PATH="/opt/jre/bin:${PATH}"
COPY --from=builder /jre /opt/jre
WORKDIR /application
# the least changing layer first: a new build of the application reuses the cached
↪ dependency layer
```

```

COPY --from=builder /builder/extracted/dependencies/ ./
COPY --from=builder /builder/extracted/spring-boot-loader/ ./
COPY --from=builder /builder/extracted/snapshot-dependencies/ ./
COPY --from=builder /builder/extracted/application/ ./
USER spring
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --start-period=30s --retries=3 \
  CMD wget -qO- http://localhost:8080/actuator/health/liveness || exit 1
# exec form: java is PID 1 and receives SIGTERM from "docker stop" → graceful shutdown
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "application.jar"]

```

- **Root olmayan kullanıcı:** Bir saldırgan süreci ele geçirirse container'da root olmaz.
- **Katman sırası:** En az değişen katman önce, böylece bir kod değişikliği ön bellekteki bağımlılık katmanını yeniden kullanır.
- **Exec formu** (["java", ...]): java PID 1'dir ve docker stop'un SIGTERM'ini alır. Shell formunda PID 1 bir shell olabilir ve sinyali iletmeyebilir.
- **-XX:MaxRAMPercentage=75:** JVM heap'ini host'un belleğinden değil container'ın bellek limitinden boyutlandırır.
- **HEALTHCHECK:** Docker (ve Compose) container'ı yalnızca liveness cevap verdiğinde healthy gösterir.

.dockerignore, Docker daemon'una target/ klasörünün tamamını değil yalnızca jar'ı gönderir.

image-sizes.sh aynı uygulamanın üç varyantını build eder:

Image	Boyut
naif (Dockerfile.simple: tam JDK, fat jar, root)	399 MB
Buildpacks (spring-boot:build-image)	353 MB
çok aşamalı + jlink (Dockerfile)	99 MB

★ İpucu

amazoncorretto:27-alpine üzerinde jlink --strip-debug, objcopy'ye ihtiyaç duyar. Builder aşaması onu apk add binutils ile kurar. O olmadan jlink Cannot run program "objcopy" ile başarısız olur.

3.3 Buildpacks: Dockerfile Olmadan Bir Image

Cloud Native Buildpacks bir Java uygulamasını tanır, bir JRE seçer, katmanları oluşturur ve iyi JVM varsayılanları ayarlar (bellek hesaplayıcı, root olmayan kullanıcı). Spring Boot'un Maven eklentisi onları Docker içinde çalıştırır:

```
./mvnw -pl modules/20-docker-deployment/lesson spring-boot:build-image
```

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <name>springbootedu/docker-deployment:buildpacks</name>
      <env>
        <BP_JVM_VERSION>25</BP_JVM_VERSION>    <!-- no Java 27 in Paketo yet -->
      </env>
    </image>
  </configuration>
</plugin>
```

	Kendi Dockerfile'nınız	Buildpacks
Kontrol	her satır	ortam değişkenleri üzerinden (BP_*)
Bakım	base image'ları siz güncellersiniz	yeniden build edin, builder yamaları getirir
Boyut	jlink ile en küçük	daha büyük (tam JRE, araçlar)
Java sürümü	kurabildiğiniz her şey	yalnızca buildpack'in sunduğu

3.4 Health Probe'ları ve Zarif Kapanış

```
server:
  shutdown: graceful          # the default since Boot 3.4 – written here to
  ↪ make it visible
management:
  endpoints:
    web:
      exposure:
        include: health,info
  endpoint:
    health:
      probes:
        enabled: true         # /actuator/health/liveness and /readiness (the
  ↪ default in Boot 4)
```

```

group:
  readiness:
    include: readinessState,db      # not ready without the database ...
  liveness:
    include: livenessState          # ... but still alive: a restart would not bring the
↪ database back

```

Uygulama readiness'ını kendisi de değiştirebilir, örneğin bir ön belleği ısıtırken. HealthProbesTest uygulamanın canlı kaldığını gösterir:

```

@Test
void anApplicationThatRefusesTrafficIsAliveButNotReady() {
    AvailabilityChangeEvent.publish(context, ReadinessState.REFUSING_TRAFFIC);    // e.g.
↪ during a long cache warm-up
    try {
        http.get().uri("/actuator/health/readiness").exchange()
            .expectStatus().isEqualTo(503)
            .expectBody().jsonPath("$.status").isEqualTo("OUT_OF_SERVICE");
        http.get().uri("/actuator/health/liveness").exchange().expectStatus().isOk();
    } finally {
        AvailabilityChangeEvent.publish(context, ReadinessState.ACCEPTING_TRAFFIC);
    }
}

```

Zarif kapanış: SIGTERM'de web sunucusu yeni istekleri kabul etmeyi bırakır ve çalışan istekler bitebilir. Sınır burada ayarlanır:

```

lifecycle:
  timeout-per-shutdown-phase: 20s      # how long running requests may still take after
↪ SIGTERM

```

Deneyin: /api/slow?millis=5000'i çağırın ve o çalışırken docker stop'u çalıştırın. İstek 5 saniye sonra yine 200 döner, ardından container durur. GracefulShutdownTest aynısını Docker olmadan kanıtlar: Bir istek çalışırken uygulama context'ini kapatır. Platform süreci öldürmeden önce bu süreden daha uzun beklemelidir (Compose'da stop_grace_period, Kubernetes'te terminationGracePeriodSeconds, modül 22).

3.5 Ortamdan Yapılandırma

```

@ConfigurationProperties("bookstore.shop")
public record ShopProperties(@DefaultValue("Bookstore") String name,
↪ @DefaultValue("TRY") String currency) {
}

```

Boot'un **relaxed binding**'i ortam değişkenlerini property'lere çevirir: BOOKSTORE_SHOP_NAME → bookstore.shop.name, SPRING_DATASOURCE_URL → spring.datasource.url. Aynı image her ortamda çalışır. Yalnızca ortam değişkenleri farklıdır. ShopPropertiesTest bu tür değişkenleri uygulamayı başlatmadan bağlar.

! Uyarı

Sırlar (parolalar, token'lar) image'daki application.yaml'dan değil, ortamdan veya bir secret deposundan gelir. compose.yaml `${POSTGRES_PASSWORD:-bookstore}` kullanır: Varsayılan değer yalnızca yerel geliştirme içindir.

3.6 Docker Compose: Uygulama ve Veritabanı

```
services:
  postgres:
    image: pgvector/pgvector:0.8.6-pg18
    environment:
      POSTGRES_DB: bookstore
      POSTGRES_USER: bookstore
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-bookstore} # dev default only; set it in
↪ the environment
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U bookstore -d bookstore"]
      interval: 5s
      timeout: 3s
      retries: 10
      # no "ports": only the app can reach the database, over the compose network

  app:
    build: lesson # lesson/Dockerfile
    image: springbootedu/docker-deployment:jlink
    ports:
      - "8080:8080"
    environment: # 12-factor: config from the
↪ environment
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/bookstore
      SPRING_DATASOURCE_USERNAME: bookstore
      SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD:-bookstore}
      BOOKSTORE_SHOP_NAME: ${BOOKSTORE_SHOP_NAME:-Bookstore on Compose}
    depends_on:
      postgres:
        condition: service_healthy # start only when the database
↪ answers
    stop_grace_period: 30s # longer than
↪ spring.lifecycle.timeout-per-shutdown-phase
```

```
./mvnw -pl modules/20-docker-deployment/lesson package  
docker compose -f modules/20-docker-deployment/compose.yaml up --build
```

- Servisler compose ağında birbirlerini adla bulur (postgres:5432).
- Veritabanı hiçbir port yayınlamaz: Ona yalnızca uygulama ulaşabilir.
- condition: service_healthy ile depends_on, uygulamayı yalnızca pg_isready cevap verdiğinde başlatır.

DockerComposeIT, package'tan sonra çalışır: Bu compose dosyasını Testcontainers ile (ComposeContainer) başlatır, bu da Dockerfile'ı build eder, readiness'ı bekler ve API'yi çağırır.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Veritabanını (veya başka bir bağımlılığı) asla liveness probe'una koymayın. Bir veritabanı kesintisi her örneği bir döngü içinde yeniden başlatır.

- **Yapın:** ENTRYPOINT için exec formunu kullanın ve platforma spring.lifecycle.timeout-per-shutdown-phase'den uzun bir bekleme süresi verin.
- **Yapmayın:** Container'ları root olarak çalıştırmayın veya çalışma zamanı image'ına build araçları (bir JDK, Maven) koymayın.
- **Yapın:** Katmanları nadiren değişenden sık değişene sıralayın ve build context'ini .dockerignore ile küçük tutun.
- **Yapmayın:** Ortama özgü değerleri veya sırları image'a gömmeyin. Onları ortam değişkenleri olarak verin.
- **Yapın:** Base image sürümlerini sabitleyin (alpine:3.22, amazoncorretto:27-alpine) ve güvenlik yamaları için düzenli olarak yeniden build edin.
- **Yapmayın:** depends_on'u koşulsuz kullanmayın. Yalnızca container başlayana kadar bekler, veritabanı cevap verene kadar değil.

5. Özet

- Çok aşamalı bir Dockerfile build'i çalıştırmadan ayırır: Katmanlı jar, jdeps + jlink ile yapılmış bir JRE, root olmayan bir kullanıcı. Burada: 399 MB yerine 99 MB.
- Buildpacks, boyut ve Java sürümü üzerinde daha az kontrolle, Dockerfile olmadan iyi bir image build eder.
- Liveness "beni yeniden başlat?", readiness "bana trafik gönder?" diye sorar. Bağımlılıklar yalnızca readiness'a aittir.
- Zarif kapanış, çalışan isteklerin SIGTERM'den sonra timeout-per-shutdown-phase içinde bitmesini sağlar.

- Tüm ortamlar için tek image: Relaxed binding ile ortam değişkenlerinden yapılandırma.
- Docker Compose uygulamayı ve bağımlılıklarını, health check'lerle doğru sırada başlatılmış olarak birlikte çalıştırır.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Boot — Dockerfiles](#) · [Cloud Native Buildpacks](#)
- [Spring Boot — Graceful Shutdown](#)
- [Spring Boot — Application Availability](#)
- [Docker — Multi-stage builds](#) · [Compose startup order](#)
- [The Twelve-Factor App](#)